

DATABASE MANAGEMENT SYSTEMS

Warehouse Inventory and Stock Management Database

TAE-II • SQL Implementation & Advanced Database Operations

Presented by: Kanchan Manohar Rana Roll No.: P17

Class: T.Y. B.Tech CSE (Data Science) Subject: Database Management Systems

RDBMS: MySQL Academic Year: 2026–27

Guided By: Mr. Chinmay Mukim

From raw warehouse data to a structured relational database with SQL querying,
database programming and query optimization.

Problem Statement & Objectives

Problem Statement:

Warehouse operations involve procurement, inventory, sales, dispatch and stock movement. Manual or spreadsheet-based tracking can make stock counts inconsistent, reorder points easy to miss, and transaction history difficult to connect.

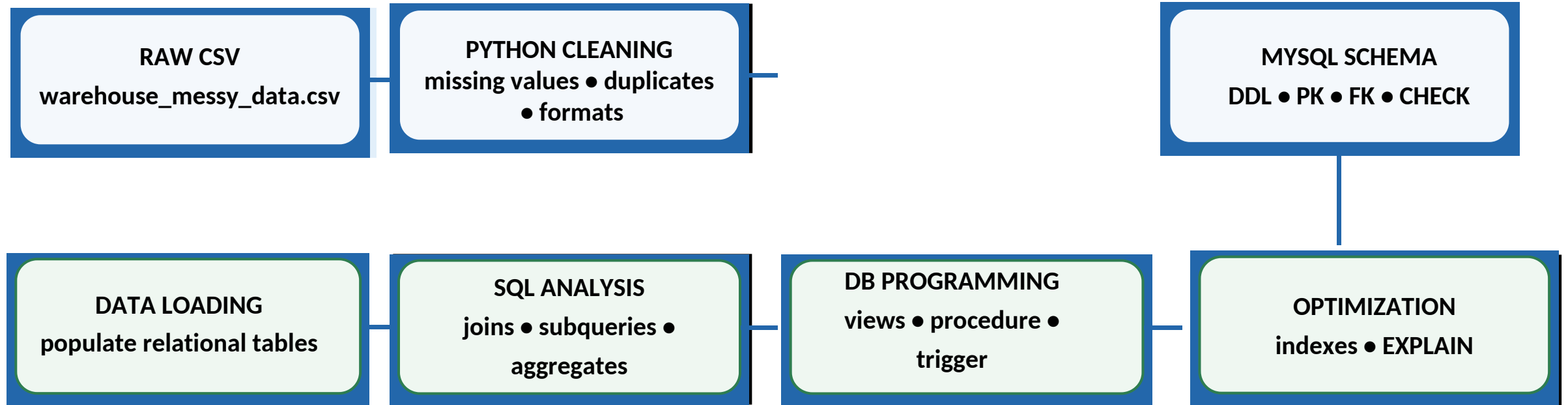
System Goal:

Design a relational database that keeps inventory, procurement and sales information consistent, auditable and queryable across multiple warehouses.

OBJECTIVES

- Clean and prepare warehouse data for database loading.
- Translate the relational design into executable MySQL DDL.
- Maintain PK/FK constraints and data integrity.
- Support procurement, sales, inventory and shipment transactions.
- Implement joins, subqueries and aggregate reporting.
- Create reusable views, a stored procedure and a trigger.
- Add indexes to frequently filtered/joined columns.
- Validate query plans using EXPLAIN / EXPLAIN ANALYZE.

Complete Project Pipeline



Dataset & Data Cleaning

Input Dataset:

Warehouse data begins as a messy CSV. The project uses Python cleaning before the records are loaded into MySQL.

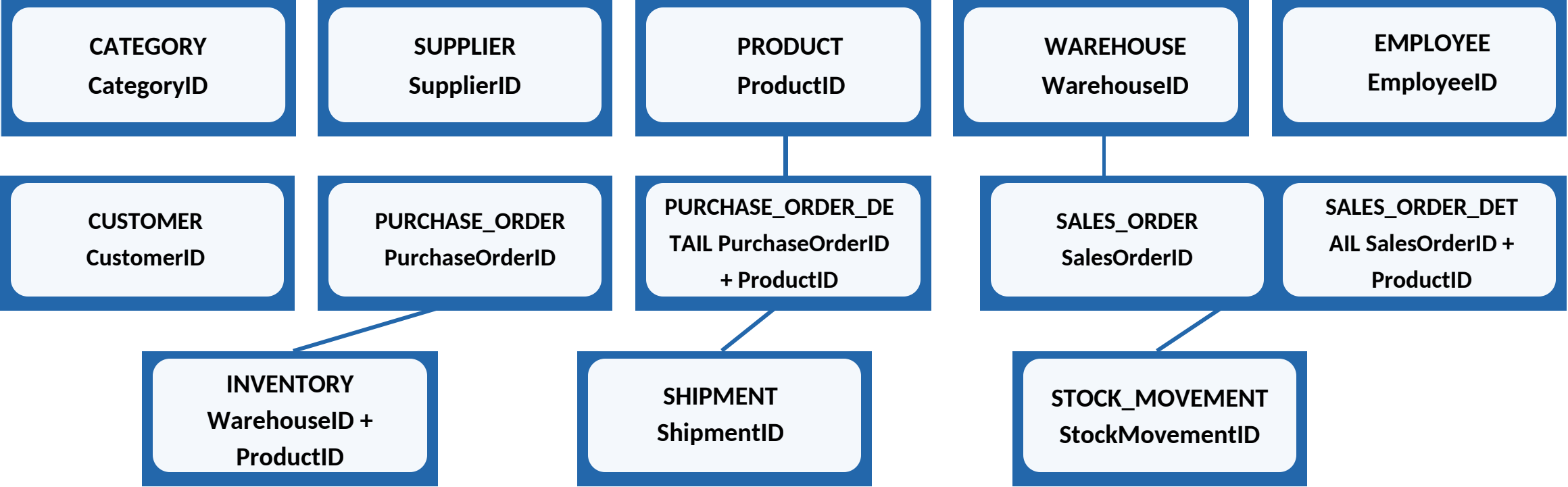
Cleaning Stage:

Typical preparation steps represented in the project flow include handling missing values, removing duplicate records, correcting data types and standardizing inconsistent values.

BEFORE → AFTER



Database Design & Relational Schema



DDL, Constraints & Data Population

DDL / Schema:

CREATE TABLE statements translate the relational model into MySQL. Primary keys, foreign keys, CHECK constraints and DEFAULT clauses enforce integrity.

Data Population:

Cleaned records are loaded into the relational tables while preserving key relationships and transaction history.

Important Constraints:

PK • FK • NOT NULL • UNIQUE •
CHECK • DEFAULT • ON
DELETE/UPDATE actions

```
CREATE TABLE Product (  
  ProductID INT PRIMARY KEY  
  AUTO_INCREMENT, ProductName  
  VARCHAR(120) NOT NULL,  
  SKU VARCHAR(30) UNIQUE NOT NULL,  
  CategoryID INT NOT NULL,  
  UnitPrice DECIMAL(10,2) NOT NULL,  
  ReorderLevel INT DEFAULT 10,  
  FOREIGN KEY (CategoryID)  
  REFERENCES Category(CategoryID)  
);
```

Implementation checklist:

- Create database and tables in dependency-safe order.
- Load master tables before transaction/detail tables.
- Validate FK relationships after import.
- Verify row counts and sample records in Workbench.

Advanced SQL Queries

JOIN Queries:

Use related tables to combine product, warehouse, customer, order and inventory information. The implementation covers Inner, Left, Cross and Self joins.

Correlated Subqueries:

Subqueries reference values from the outer query and can be combined with aggregates, GROUP BY and HAVING for business-level analysis.

Aggregate Reporting:

SUM, COUNT, AVG and related aggregation can support sales, inventory and supplier-performance reports.

Example Analysis:

Low-stock reporting compares Inventory.QuantityOnHand with Product.ReorderLevel to identify products that need attention.

Views, Stored Procedure & Trigger

VIEWS:

Reusable database-level representations for reporting and simplified access.

Code:

```
CREATE VIEW inventory_report AS
SELECT product_id, product_name,
quantity
FROM inventory
WHERE quantity > 0;
```

STORED PROCEDURE:

Encapsulates a repeatable business operation inside MySQL.

Code:

```
DELIMITER //
CREATE PROCEDURE receive_goods(
    IN p_id INT,
    IN p_qty INT
)
BEGIN
    UPDATE inventory
    SET quantity = quantity + p_qty
    WHERE product_id = p_id;
END //
DELIMITER ;
```

TRIGGER:

Automatically executes when a specified database event occurs.

Code:

```
CREATE TABLE inventory_audit
(
    product_id INT,
    old_qty INT,
    new_qty INT
);
```

Results, Conclusion & Future Scope

IMPLEMENTATION OUTCOME:

- Raw warehouse data is transformed into a structured relational design.
- The schema supports procurement, sales, inventory, shipment and stock-movement operations.
- Advanced SQL demonstrates multi-table analysis and reusable database logic.
- Indexes and EXPLAIN provide a basis for query-performance validation.

FUTURE SCOPE:

- Automated ETL/data-loading pipeline
- Dashboard and reporting integration
- Role-based security and application layer
- Cloud database deployment and monitoring

CONCLUSION:

Project Conclusion

The project demonstrates a complete DBMS workflow: data preparation → relational modelling → MySQL implementation → SQL analysis → database programming → performance optimization.

REFERENCES:

TAE-IDatabase DesignProposal • Warehouse Inventory and Stock Management Database
MySQL 8.x implementation roadmap • Project repository supplied with the project

Thank You!!!